REPORT

All the experiments were carried out on the **Linux** machines of the school, and we tried to run the experiments hours when there was no big load, which we controlled with the script workload.sh that we have attached to the project.

Indicatively, a machine has:

Number of cores : 4

Processor : Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

OS : NAME="Ubuntu" VERSION="20.04.6 LTS (Focal Fossa)"

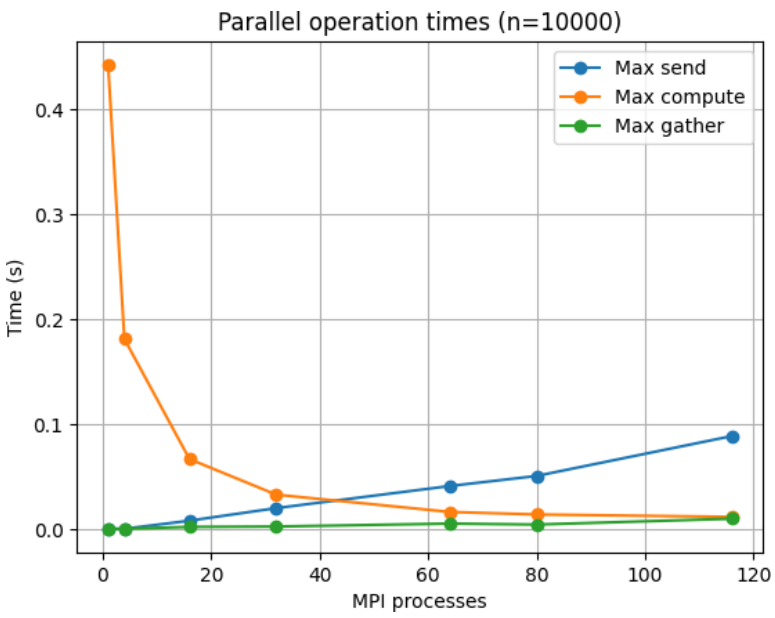
Compiler Version : 9.4.0

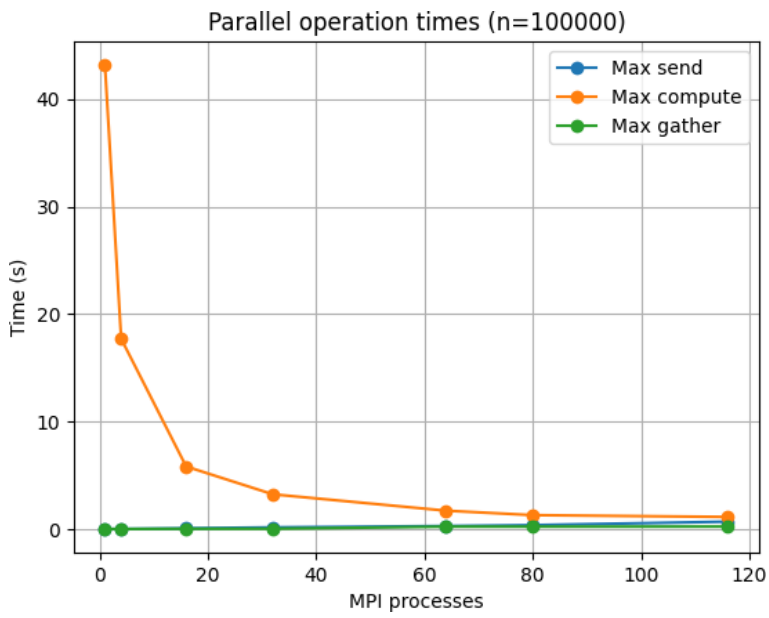
Exercise 3.1

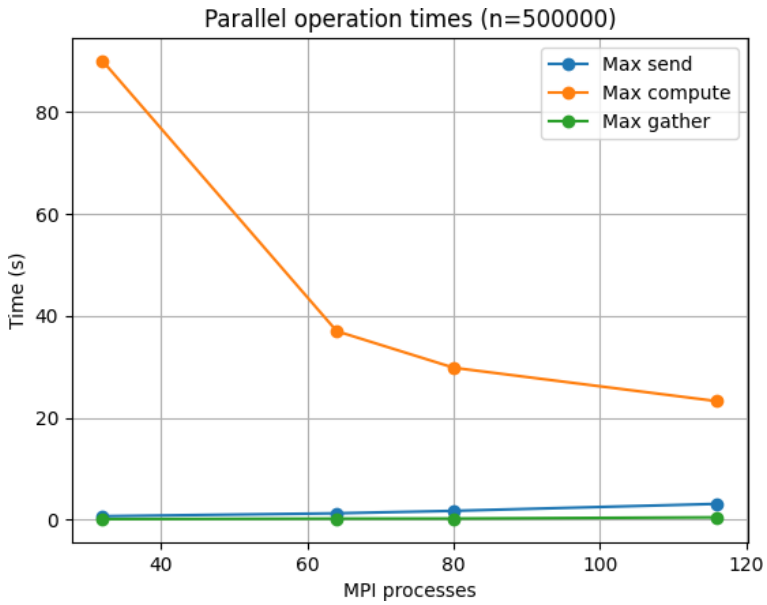
In the polynomial multiplication exercise, initially process 0 makes the polynomials and sends to the rest of the processes. This is calculated with the following logic:

If a process calculates result points greater than the degree of the input polynomial (even 4th degree polynomials, and each process calculates a coefficient of the result at a time), then it does not need any "small coefficients" because with whichever coefficient it is multiplied by the other polynomial, they will not bring the result we want as a force. For example, if a process calculates x^7 in the previous example, x0, x^1, x2 are not needed because they cannot be give a coefficient of x7 as a result even if it is multiplied by the maximum input factor x^4. Similarly, in the case where a process calculates small coefficients e.g. x^2, x3 , x 4 is not needed.

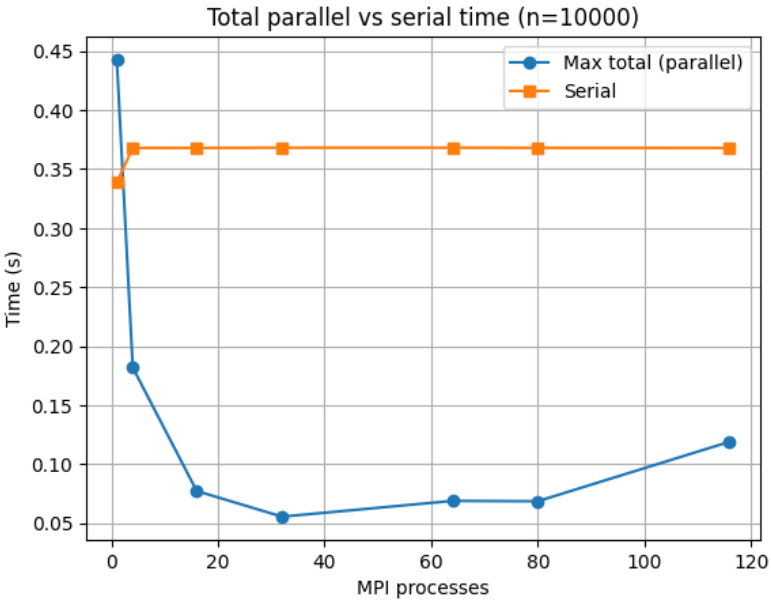
Also, we have put an extra parameter in the program if the user gives the word **no** it means that the serial algorithm will not run (we did it for some tests that we wanted to force the parallel algorithm a lot but the serial would never end).

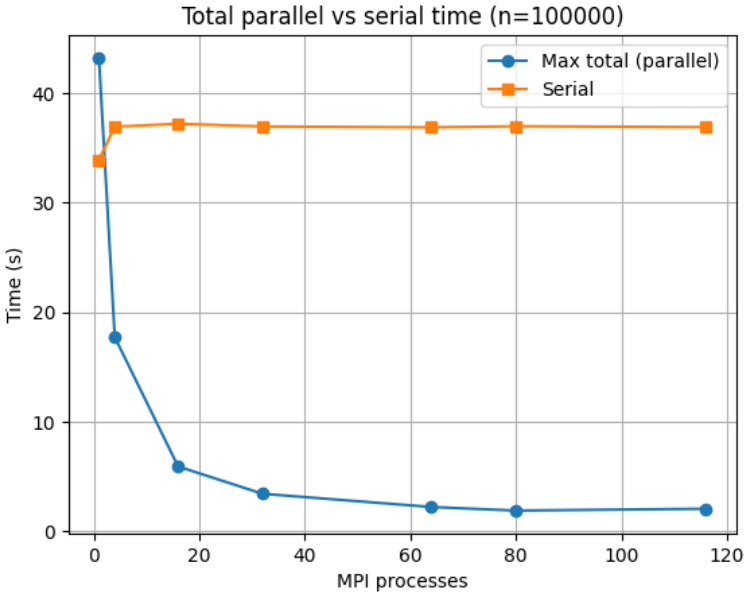


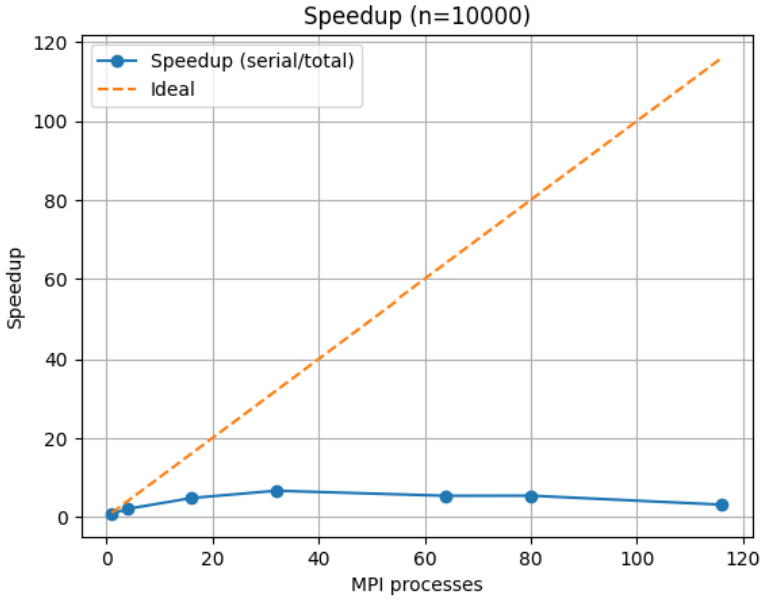


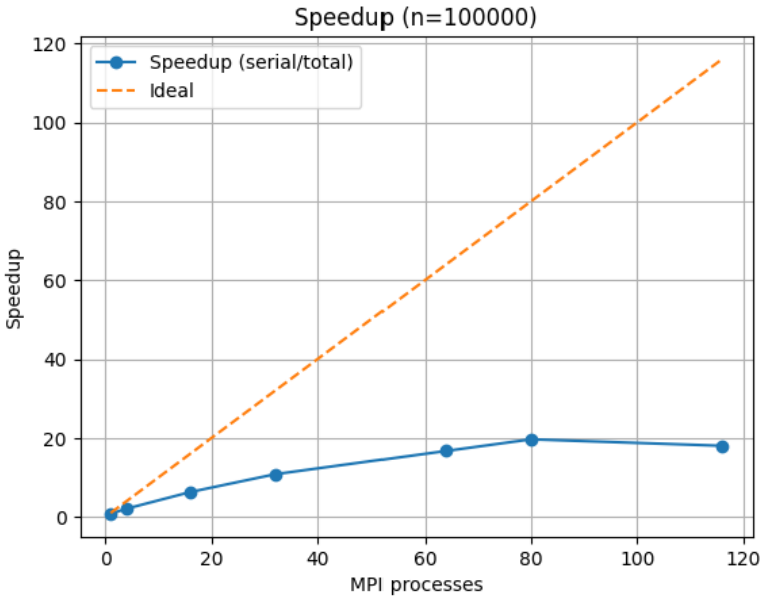


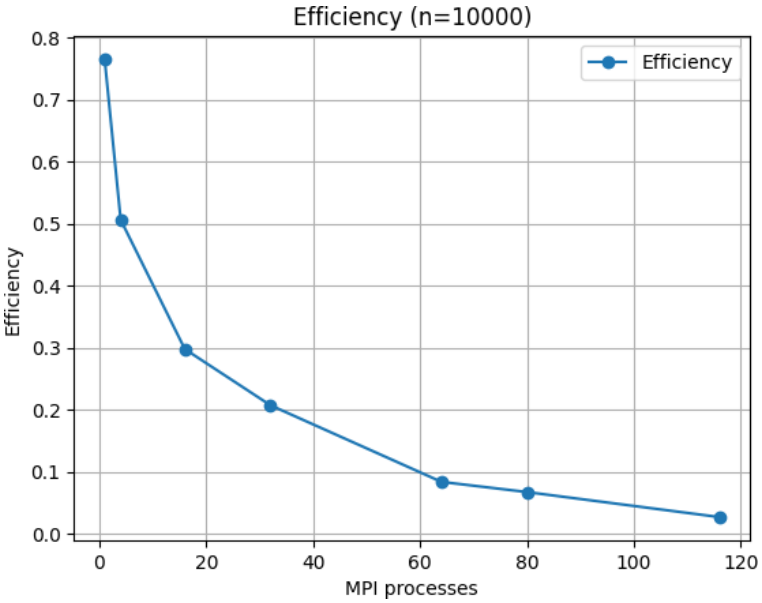
In the comparison of the parallel algorithm for a different degree of polynomial we see that the time for compute decreases asymptotically up to 116 processes (we put up to 29 machines on 4 processes so that we can clearly see the results on the diagram) and the work is divided among the processes. Also, we see that for 5* 105 there is a more noticeable reduction from 80 to 116 processes precisely because there is a workload for too many processes. As for gather and send times
are low on computation. But we see (especially in the diagram with the least degree of polynomial) that in many Proccess it rises
significantly in the 116 processes, which makes sense since data has to be sent to many processes.

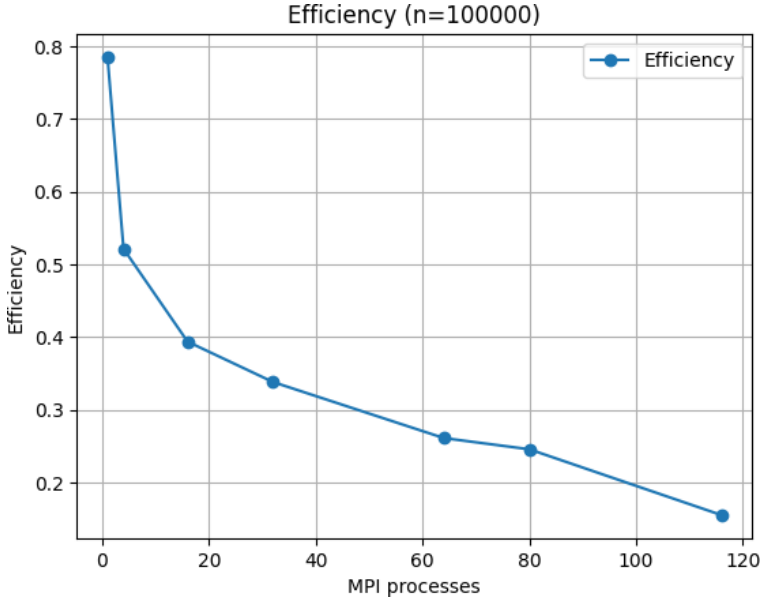

Total parallel vs serial time (n=10000)
Max total (parallel)
Serial
Time (s)
MPI processes


Total parallel vs serial time (n=100000)
Max total (parallel)
Serial
Time (s)
MPI processes


Speedup (n=10000)
Speedup (serial/total)
Ideal
Speedup
MPI processes


Speedup (n=100000)
Speedup (serial/total)
Ideal
Speedup
MPI processes


Efficiency (n=10000)
Efficiency
Efficiency
MPI processes


Efficiency (n=100000)
Efficiency
Efficiency
MPI processes

In the comparison of a serial with a parallel algorithm we see that for only 1 proccess the serial is faster (logical since there is also the overhead of sending and collecting data in parallel). We noticed that the
Serial algorithm runs slightly faster for 1 process and this is explained because when there is only one process on one machine it is faster than with 4 on the same machine (lack of competition for CPU
, scheduling, etc.). In addition, we see again that for n = 10^5 there is no
Worse performance for max processes while for a lesser extent it exists. We see that the speedup is relatively low (especially for many processes) since there is also communication in the middle and we do not gain so much from more and more processes. This is also evident from the asymptotic course of efficiency.

Χρονοι με openMP

Threads	$n = 10^4$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0005	0.3455	0.3873	0.3492	0.9900
2	0.0007	0.3537	0.1804	1.9600	98.010
4	0.0006	0.3454	0.0983	3.5160	87.844
8	0.0008	0.3493	0.1000	3.4940	43.684

Threads	$n = 10^5$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0041	34.032	34.679	0.9800	98.134
2	0.0071	34.013	17.876	1.9000	95.132
4	0.0042	34.117	9.7804	3.4880	87.208
8	0.0072	34.076	9.6007	3.5500	44.368

Remembering the times from OpenMP we see that if we compare the performance on a node 4 processes with MPI and 4 threads OpenMP the times are almost twice as long with MPI and this is logical and expected since the processes are "heavier" than the threads, the context-switching is slower, they do not have shared memory, and therefore many data copies. The value of mpi can be seen when we increase the Process on different machines, then we see a real improvement up to one order of magnitude down (by 10) in
relationship with the OpenMP implementation.

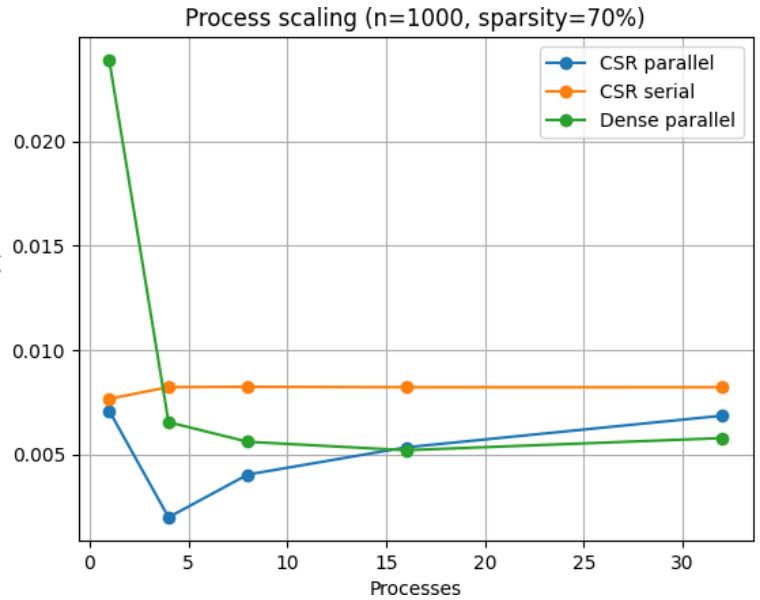
Exercise 3.2

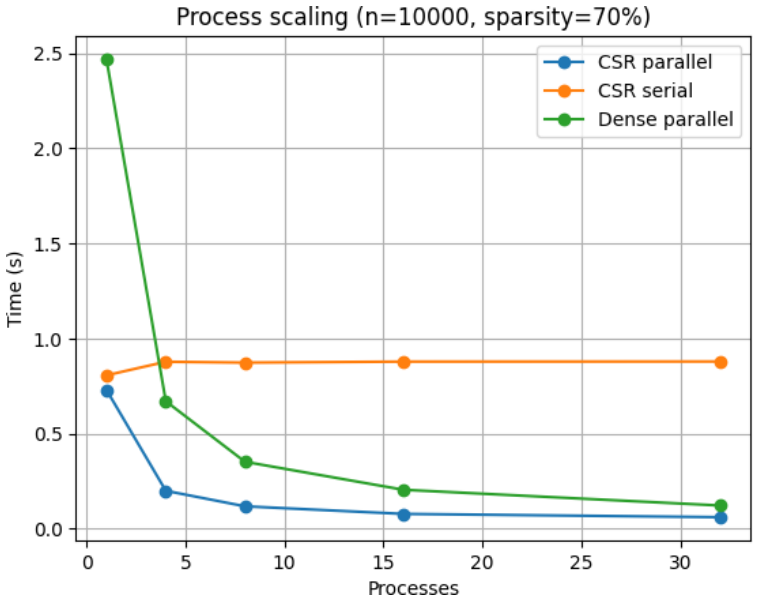
This problem presents the use of CSR in the form of arrays that greatly speed up the multiplication of a matrix by vector in sparse mainly paintings. The logic obviously follows that of the corresponding exercise in task 2 and is as follows:

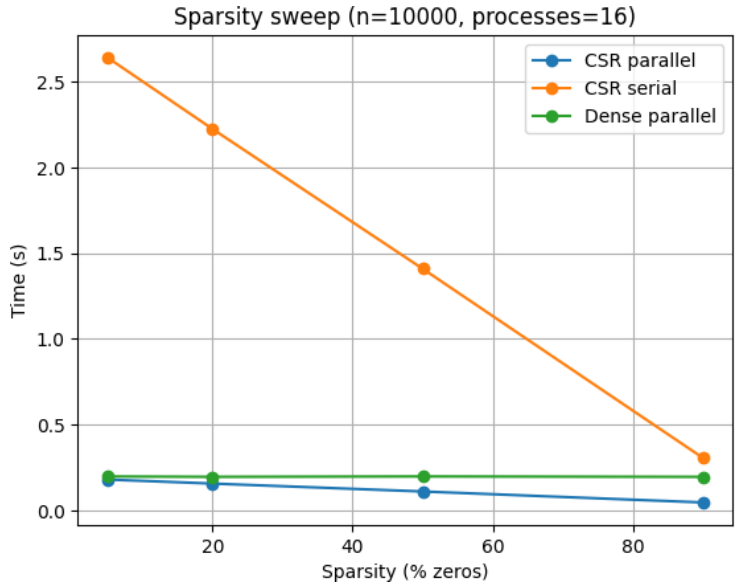
First, the array and vector are initialized with random numbers from process 0 as mentioned in the spokesman, while initializing the CSRs is also carried out only by this process, i.e. serially. In order to carry out the parallel calculation, it was necessary to compute enough data related to each process, i.e. the "local" data. Specifically, the data utilized by the processes are the three CSR tables. The non_zero_values and column_indeces tables are shared by process 0 to all the others through the MPI_Scatterv call. This call ending in v is necessary, as each process receives a different size of data. This is because even though the lines are "split" (as far as this is possible), the number of non-zero elements can vary significantly from line to line. To make these two tables scatter in all the processes it is necessary for rank = 0 to know the number of non-zero elements that each process will read (counts), as well as the total number of elements until before this process reads (offsets/displacements). In addition, each process needs to know the amount of data it will receive, so it also calculates its own local data. For the calculation of multiplication, it is necessary for everyone to know the vector and the 3rd CSR table with the row_indeces, which are broadcast. For the dense parallel algorithm, the entire array is necessarily scattered, with the difference that each process will read as many elements as its rows on the Dimension of the table (square), where it is much simpler to calculate counts and offsets for ScatterV.

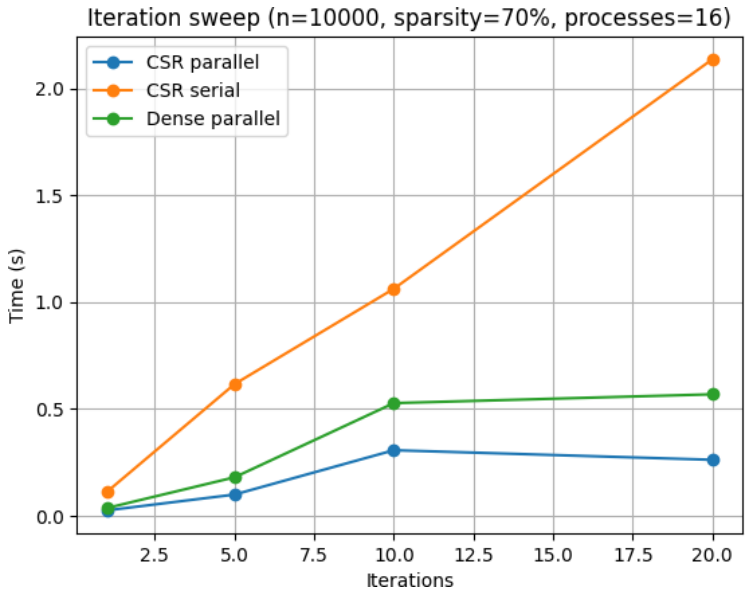
Regarding the execution of multiplication itself, the results of each process for each iteration are compiled into a local result, which at the end of each iteration is compiled from all processes using the MPI_Allgatherv, so that for the next iteration it can act as an input for all processes.

As in task 2 the parameters that remained constant when one changed were configured to: **Iterations**:10 , **Sparsity**: 70%, **Processes**: 16.
experiments ran 4 times each to gather the results.









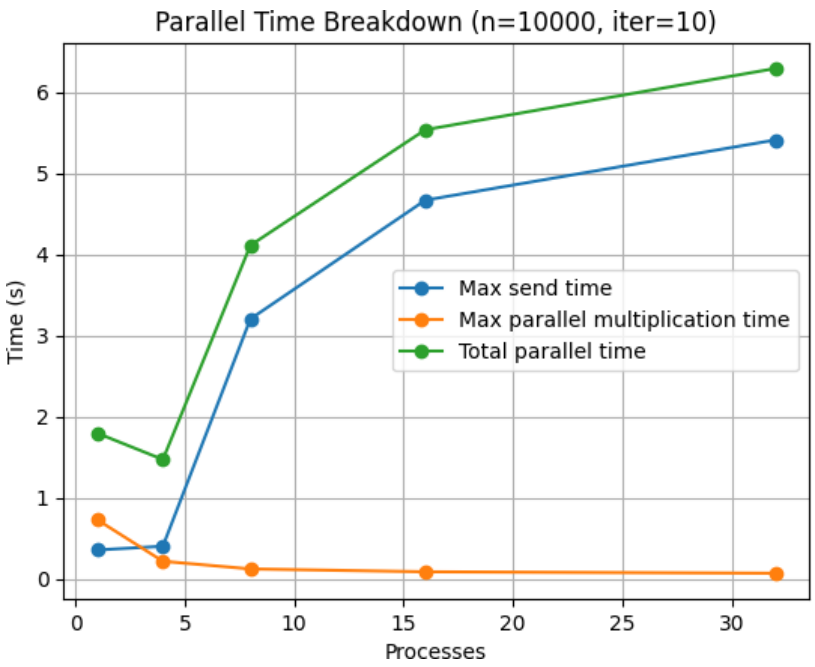
Comparing Parallel CSR to Serial:

As we can see from the images, the serial algorithm generally exhibits a smooth behavior and change as expected while the parallel is better in all diagrams.

In particular, with the increase of processes, the serial remains almost unchanged, since it is not affected by this parameter obviously (except in the case we mentioned before for 1 process). For a small array size of 1000, the parallel algorithm increases as the processes increase, as the communication and send time increases as the program executes. In short, for not so large data sizes, the parallelism is relatively overlapped by this time and its value is not highlighted as much as for larger sizes. Regarding the density of the zero elements, in a very large sparsity of zeros we see the two times converging and this is to be expected. The serial shows greater slope, as the existence of more and more zeros constitutes a smaller work to perform. In contrast to the change in iterations, we see that for many iterations the parallel does extremely well since too many processes take advantage of the workload for the most part, while the
Serial increases exponentially as expected and intuitively due to the continuous increase of executions of multiplication that need to be done.

Comparison of Parallel CSR with Dense Parallel:

The Dense Parallel algorithm starts from very high but takes advantage of the parallelism early on and decreases considerably. In fact, for many processes where communication is more costly, and for the smallest dimension number (n = 1000) the CSR parallel time exceeds dense parallel. However, for n = 10000 we see that the large parallelism and the large amount of data are exploited by CSR (let's not forget that sparsity is 70%, so since we have too much data, the performance of CSR is more pronounced method) and is the fastest throughout the duration of the diagram. With the change in sparsity, the dense algorithm remains constant, as it is expected, since he executes the same number of operations regardless of a large percentage of zero elements, since he processes the entire table, while the parallel one decreases. With regard to iterations, the two parallel arrhythmics behave similarly to dense na
It has a higher time, obviously due to the processing of all elements and not just non-zeros.



Finally, we see that the time of sending data increases a lot for many processes since the data is large and must be sent to each process and different machines (specifically from 4 -> 8 processes we see a big difference since we go from execution to 1 machine to execution to 2). In fact, it is most of the parallel CSR algorithm since the multiplication time is very short especially for many processes.